



INSTITUT INFORMATIQUE SUD AVEYRON

Reclassement professionnel

Développement web back-end

Introduction au PHP

Sommaire

1	Définition et rôle de PHP	2
2	Sites statiques et sites dynamiques	2
3	Pourquoi utiliser PHP ?	3
4	Langages client side et server side.....	3
5	Mise en place de l'environnement de travail	4
5.1	Le serveur interne de PHP	4
5.2	Logiciels recréant une architecture serveur sur une machine locale	4
5.3	L'éditeur de texte ou l'IDE	4
6	Créer, enregistrer et exécuter un script PHP	5
6.1	Où écrire notre code PHP ?	5
6.2	La balise PHP	5
6.3	Syntaxe PHP de base.....	5

1 Définition et rôle de PHP

PHP est l'acronyme de « PHP Hypertext Processor ».

Créé en 1994 par Rasmus Lerdof, sa version la plus récente à ce jour est la version 8 qui date du 26/11/2020.

Tout au long de notre apprentissage, nous utiliserons la version 7.x.x de PHP.

Le PHP est un langage qui s'exécute coté serveur (server side) et qui nous permettra entre-autre de générer dynamiquement des pages HTML.

En d'autres mots, grâce au PHP, nous allons pouvoir afficher des contenus différents sur une même page en fonction de certaines variables (heure, date, utilisateur connecté ou pas ...).

<https://fr.wikipedia.org/wiki/PHP>

2 Sites statiques et sites dynamiques

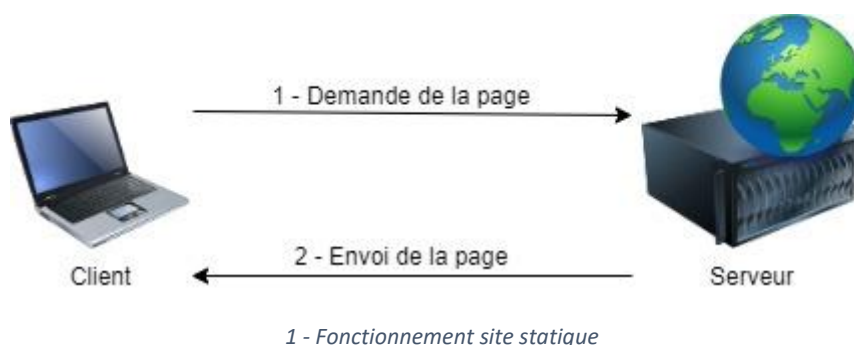
Les langages de programmation destinés au web sont catégorisés selon deux grands types :

- Langages statiques VS langages dynamiques
- Langages avec exécution coté serveur (server side) VS langages avec exécution coté client (client side)

Les sites dits statiques se caractérisent par le fait qu'aucune interaction ni adaptation ne sont possibles. Le code des différentes pages ne va pas changer en fonction de l'utilisateur ou de toute autre variable. Cela ne remet pas en question l'utilité d'un site statique. En effet, quel serait l'intérêt de réaliser un CV dynamique ?

Le code des pages statiques est stocké sur le serveur qui le restitue sans aucune manipulation.

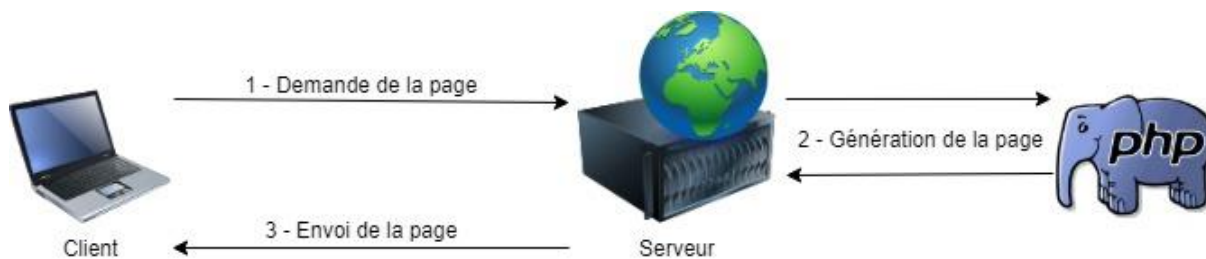
Le client demande la page et le serveur lui envoie.



A contrario, un site dynamique se caractérise par le fait qu'une même page peut contenir des informations adaptées en fonction de l'utilisateur, d'une date, d'une heure ...

Ce côté dynamique est rendu possible par l'utilisation d'un langage server side tel que PHP.

Le client demande la page, le serveur demande à PHP de la lui fabriquer puis envoi la page au client.



2- Fonctionnement site dynamique

3 Pourquoi utiliser PHP ?

Contrairement au HTML et au CSS qui sont de véritables standards, PHP à de nombreux concurrents : Python, Ruby, JavaScript, Java ...

Il n'y a aucune raison concrète pour préférer un de ces langages car ils ont tous leurs avantages et leurs inconvénients.

Alors pourquoi PHP ?

- La structure du langage le rend accessible aux débutants qui pourront rapidement obtenir des résultats. Le PHP va supporter aussi des structures très complexes.
- C'est est un langage Open Source et donc gratuit.
- Il est largement supporté et reconnu et va donc pouvoir fonctionner avec l'immense majorité des architectures techniques.
- PHP se distingue par ses performances et sa solidité.
- Son statut Open Source et son âge font qu'une immense communauté de développeurs existe et qu'il est donc aisé de trouver de la documentation et de l'aide.
- Cette très grande communauté permet aussi de corriger rapidement les rares failles de sécurités découvertes.

4 Langages client side et server side

Un site internet est un ensemble de fichiers de codes liés entre eux et faisant éventuellement appel à des ressources ou médias comme des images, ect.

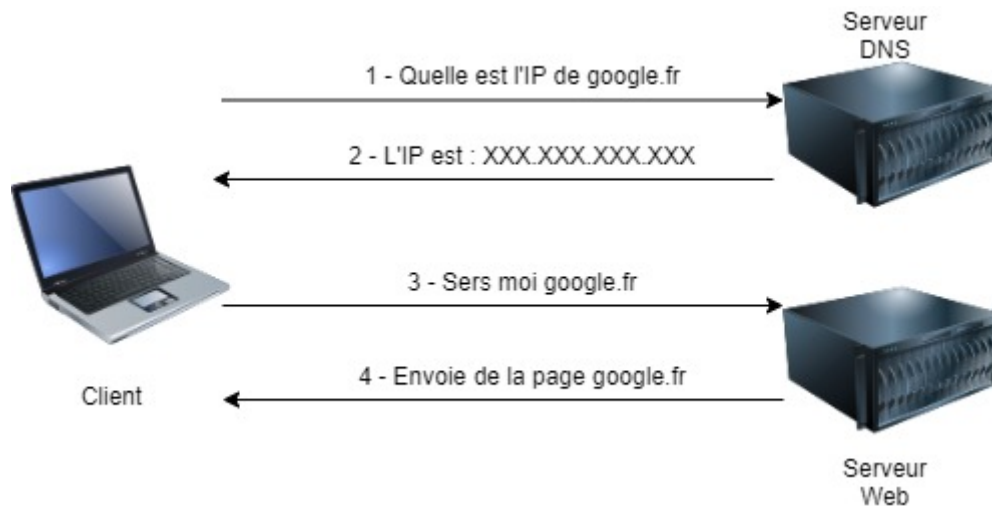
Le code écrit dans ces fichiers, encore appelé « script », va pouvoir être exécuté soit côté client (client-side), c'est-à-dire directement dans le navigateur de l'utilisateur qui cherche à afficher la page, soit du côté du serveur (server-side).

La grande majorité des navigateurs n'est capable de comprendre et d'exécuter que du HTML, du CSS et du JavaScript. Un navigateur est ainsi incapable de comprendre un script PHP.

Si une page demandée par le visiteur ne contient aucun code nécessitant l'intervention du serveur, ce dernier va renvoyer (servir) la page telle quelle et le navigateur va l'afficher. C'est ce qui se passe avec une page HTML.

Dans le cas d'une page contenant du code à interpréter par le serveur, celui-ci va construire la page avant de la servir. A noter, que le code présent dans une page n'est jamais visible par le client et que les opérations sont transparentes pour le client.

Le schéma ci-dessous résume ce qu'il se passe lorsqu'une page est demandée via un navigateur.



3- Demande d'une page à un serveur

5 Mise en place de l'environnement de travail

Pour coder en HTML et CSS, un éditeur de texte et un navigateur suffisent. Pour coder des pages en PHP, cela va être un peu différent car il va falloir reproduire un environnement coté serveur.

Nous allons découvrir ici les différentes alternatives.

5.1 Le serveur interne de PHP

PHP est livré avec un [serveur interne](#) à des fins de développement. Ce serveur n'est pas utilisable en production.

5.2 Logiciels recréant une architecture serveur sur une machine locale

Différents logiciels sont disponibles afin de recréer un environnement serveur.

- Wamp, EasyPHP, Laragon pour Windows.
- Mamp pour Mac OS.
- Xamp pour Linux.

Nous travaillerons avec [Laragon](#) et ce choix est tout à fait arbitraire.

5.3 L'éditeur de texte ou l'IDE

Là encore, pléthores d'éditeurs sont disponibles mais afin de simplifier les choses, nous utiliserons [Visual Studio Code](#). Ce choix est tout aussi arbitraire que le précédent.

6 Créer, enregistrer et exécuter un script PHP

A partir de maintenant, nous allons véritablement rentrer dans le vif du sujet et commencer à pratiquer.

6.1 Où écrire notre code PHP ?

Nous pouvons écrire nos scripts PHP soit dans des fichiers dédiés (fichiers ne contenant que du code PHP), soit dans des fichiers HTML.

Dans les deux cas, nos fichiers devront être enregistrés avec l'extension `.php`.

6.2 La balise PHP

Pour être en mesure d'exécuter du code PHP, le serveur va devoir le reconnaître. Pour indiquer que du code est du PHP il doit être entouré de balise qui a la forme suivante : `< ?php ?>`.

Le code PHP peut être intégré à n'importe quel endroit du fichier.

```
<?php
?>
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Cours PHP</title>
  <?php ?>
</head>
<body>
  <h1>Titre principal</h1>
  <?php

  ?>
</body>
</html>
```

6.3 Syntaxe PHP de base

Voici un premier exemple de code PHP.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Cours PHP</title>
</head>
<body>
  <h1>Titre principal</h1>
  <?php
    // Affiche "Hello World" avec un retour à la ligne
    echo 'Hello World <br>'; // Commentaire PHP

    /* Affiche
```

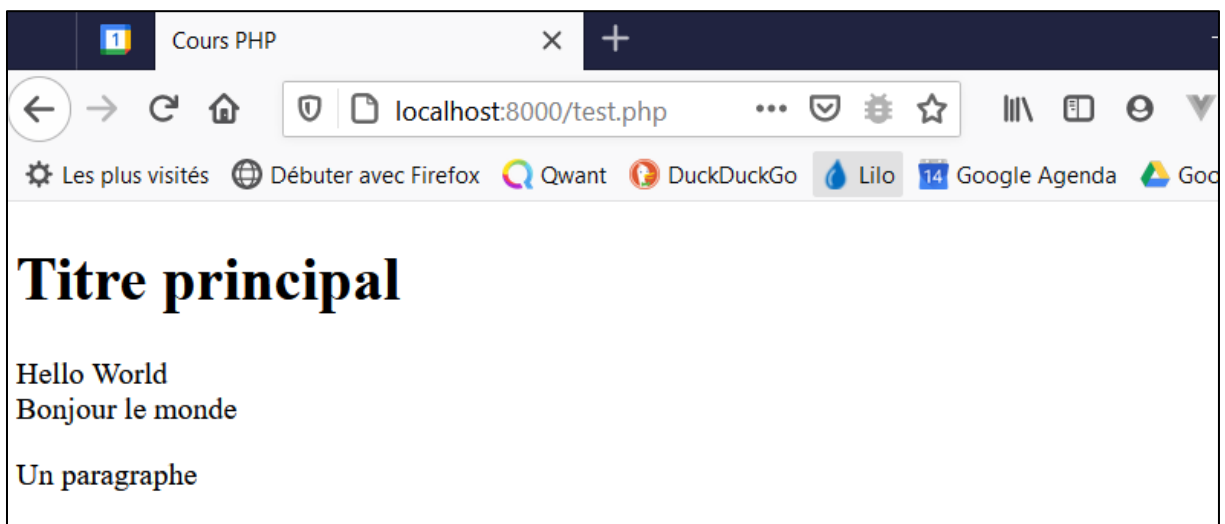
```
"Bonjour le monde"
*/
echo "Bonjour le monde"; /* Commentaire PHP */
?>
<p>Un paragraphe</p>
</body>
</html>
```

La première ligne est un commentaire PHP. En PHP, nous pourrions écrire des commentaires monoligne avec `//` et multiligne avec `/* */`.

Les parties `echo 'Hello World <br>';` et `echo "Bonjour le monde";` sont ce que l'on appelle des instructions car on demande au code de faire quelque chose.

Une instruction PHP doit toujours se terminer par un `;`. S'il est oublié, votre code ne fonctionnera pas.

Si nous affichons la page, voici le résultat :



Vous remarquerez la barre d'adresse qui contient `localhost:8000/test.php`. `localhost` est équivalent à `127.0.0.1` qui correspond à l'adresse de la machine hôte, `:8000` est le port utilisé et `test.php` est le nom du fichier à afficher.